



CSE 1112: Structured Programming Language Laboratory

Sidratul Tanzila Tasmi

Lecturer, Department of Computer Science and Engineering
United International University

Introduction^[Ref: <https://www.geeksforgeeks.org/multidimensional-arrays-in-c/>]

- A multi-dimensional array can be termed as an array of arrays that stores homogeneous data in tabular form.
- The general form of declaring N-dimensional arrays is shown below:

`data_type array_name[size1][size2]....[sizeN];`

- `data_type`: Type of data to be stored in the array.
- `array_name`: Name of the array.
- `size1, size2,..., sizeN`: Size of each dimension.

Introduction

Examples:

2-D Array

	Column 0	Column 1	Column 2
Row 0	a[0][0]	a[0][1]	a[0][2]
Row 1	a[1][0]	a[1][1]	a[1][2]
Row 2	a[2][0]	a[2][1]	a[2][2]
Row 3	a[3][0]	a[3][1]	a[3][2]
Row 4	a[4][0]	a[4][1]	a[4][2]

3-D Array

56	9	11
18	23	2
8	10	41

Two dimensional array: `int two_d[10][20];`

Three dimensional array: `int three_d[10][20][30];`

Size of Multidimensional Arrays:

- The total number of elements that can be stored in a multidimensional array can be calculated by multiplying the size of all the dimensions.
- For example:
 - The array `int x[10][20]` can store total $(10 \times 20) = 200$ elements.
 - Similarly array `int x[5][10][20]` can store total $(5 \times 10 \times 20) = 1000$ elements.

Two-Dimensional Array in C

- We can visualize a two-dimensional array as an array of one-dimensional arrays arranged one over another forming a table with 'x' rows and 'y' columns where the row number ranges from 0 to (x-1) and the column number ranges from 0 to (y-1).

	Column 0	Column 1	Column 2
Row 0	<code>x[0][0]</code>	<code>x[0][1]</code>	<code>x[0][2]</code>
Row 1	<code>x[1][0]</code>	<code>x[1][1]</code>	<code>x[1][2]</code>
Row 2	<code>x[2][0]</code>	<code>x[2][1]</code>	<code>x[2][2]</code>

Graphical Representation of Two-Dimensional Array of Size 3 x 3

Declaration of Two-Dimensional Array

- The basic form of declaring a 2D array with x rows and y columns in C is shown below.

```
data_type array_name[x][y];
```

- We can declare a two-dimensional integer array say 'x' with 10 rows and 20 columns as:

```
int x[10][20];
```

Initialization of Two-Dimensional Arrays

1.Using Initializer List:

We can initialize a 2D array in C by using an initializer list as shown in the example below.

First Method:

```
int x[3][4] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11}
```

The above array has 3 rows and 4 columns. The elements in the braces from left to right are stored in the table also from left to right.

The elements will be filled in the array in order: the first 4 elements from the left will be filled in the first row, the next 4 elements in the second row, and so on.

Initialization of Two-Dimensional Arrays

Second Method (better):

```
int x[3][4] = {{0,1,2,3}, {4,5,6,7}, {8,9,10,11}};
```

This type of initialization makes use of nested braces. Each set of inner braces represents one row.

In the above example, there is a total of three rows so there are three sets of inner braces. The advantage of this method is that it is easier to understand.

Initialization of Two-Dimensional Arrays



2. Initialization of 2D array using Loops:

We can use any C loop to initialize each member of a 2D array one by one as shown in the below example.

Example:

```
int x[3][4];  
for(int i = 0; i < 3; i++){  
    for(int j = 0; j < 4; j++){  
        x[i][j] = i + j;  
    }  
}
```

Accessing Elements of Two-Dimension Array



Elements in 2D arrays are accessed using row indexes and column indexes. Each element in a 2D array can be referred to by:

array_name[i][j]

where, i: The row index, j: The column index.

Example: ***x[2][1];***

The above example represents the element present in the third row and second column.

Accessing Elements of Two-Dimension Array



To access all the elements of a 2D array, we can use nested for loops. One to traverse the rows and the other to traverse the columns.

Output

```
/tmp/U0QICoRzCT.o
Element at x[0][0]: 0
Element at x[0][1]: 1
Element at x[1][0]: 2
Element at x[1][1]: 3
Element at x[2][0]: 4
Element at x[2][1]: 5
```

```
#include <stdio.h>
int main(void)
{
    // an array with 3 rows and 2 columns.
    int x[3][2] = { { 0, 1 }, { 2, 3 }, { 4, 5 } };

    // output each array element's value
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 2; j++) {
            printf("Element at x[%d][%d]: ", i, j);
            printf("%d\n", x[i][j]);
        }
    }

    return 0;
}
```



Thank you!